Face the Truth: Regression Cracks the Face Puzzle
OLS, Ridge, Lasso with Feature Engineering

Liran Ma, Zhipeng Cai, and Rongrong Qian

1 Background and Motivation

Imagine a police investigation where officers are trying to narrow down suspects using video images from surveillance cameras. One crucial clue could be the person's approximate age. However, when humans estimate age from a face, their guesses are often inconsistent and can be off by many years, because subtle cues may be missed or distorted by lighting, makeup, or facial expression.

How can we make age estimation more scientific and reliable, beyond just human intuition? The answer lies in data-driven methods that can systematically learn patterns from a large number of examples. Unlike humans, who rely on subjective impressions or limited experience, a statistical model can analyze thousands of labeled face images and discover which features are truly predictive of age.

A widely used approach for this task is **regression**. In simple terms, regression is a statistical technique for predicting a continuous outcome (such as age) from measured features (like image pixels or extracted descriptors). By fitting a regression model to data, we let the computer discover the quantitative relationships between facial features and age, leading to more objective and consistent predictions.

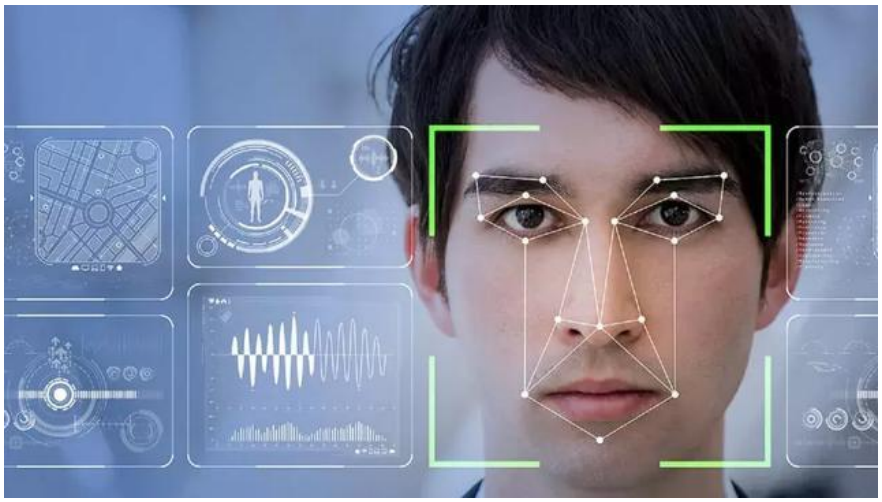

Figure 1: AI systems can automatically analyze facial features and predict attributes such as age and gender.

Why is it called regression? The term dates back to 19th-century studies by Francis Galton, who found that the heights of children tend to "regress" towards the population

average, rather than being as extreme as their parents. Today, regression refers to any technique for modeling and predicting a numerical outcome from data.

In this project, we use the UTKFace dataset to demonstrate how regression models can predict a person's age from facial images, with the goal of achieving greater accuracy than human estimation.

2 Feature Extraction and Data Processing

Before applying regression models to face images, we must first convert each image into a vector of informative features. In this project, we consider three widely used feature extraction strategies: raw pixels, Histogram of Oriented Gradients (HOG), and deep CNN features (VGG16). Each method requires its own preprocessing steps, but to ensure fair model evaluation and prevent information leakage, we always follow this workflow:

image loading → train/test split → feature extraction → standardization → regression modeling.

2.1 Raw Pixels (Flattened Images)

Process:

- **Resize** all images to a fixed size (e.g., 128×128, color or grayscale).
- **Split** the dataset into training and test sets (to prevent information leakage).
- **Flatten** each image in train and test sets into a 1D vector (e.g., $128 \times 128 \times 3 = 49{,}152$ features for color).
  Why 1D?
- **Standardize** features (zero mean, unit variance) using statistics computed from the training set, then apply the same transformation to the test set.

Example code:

```python
import numpy as np
import glob
import os
from skimage.io import imread
from skimage.transform import resize
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# ==== Set your data path here! ====
# For example: data_path = '/Users/yourname/path/to/UTKface/*.jpg'
data_path = '/path/to/your/UTKFace/*.jpg'  # <-- CHANGE this to your own folder

# 1. Load image file names
image_files = glob.glob(data_path)

ages = []
images_resized = []
img_size = (128, 128)

# 2. Load, resize, and parse age
```

```python
for file in image_files[:100]:  # Use the first 100 images for demo
    age = int(os.path.basename(file).split('_')[0])
    img = imread(file)
    img_resized = resize(img, img_size, anti_aliasing=True)
    ages.append(age)
    images_resized.append(img_resized)

ages = np.array(ages)
images_resized = np.array(images_resized)

# 3. Split into train/test (before any statistics-based processing)
X_train_img, X_test_img, y_train, y_test = train_test_split(
    images_resized, ages, test_size=0.2, random_state=42
)

# 4. Flatten images in train and test sets
X_train = X_train_img.reshape(X_train_img.shape[0], -1)
X_test = X_test_img.reshape(X_test_img.shape[0], -1)

# 5. Standardize features (fit only on train set)
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)
```

2.2 HOG Feature Extraction

Process:

- **Resize** all images to a fixed size (e.g., 128×128, color or grayscale).
- **Split** the dataset into training and test sets (to prevent information leakage).
- **Extract HOG features** for each image in the training and test sets (captures local edge and orientation patterns).
- **Standardize** features (zero mean, unit variance) using statistics computed from the training set, then apply the same transformation to the test set.

Example code:

```python
from skimage.feature import hog
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# data_path = ... (see previous example)
# 1. Load and resize images
# images_resized = ...

# 2. Split into train/test before feature extraction
X_train_img, X_test_img, y_train, y_test = train_test_split(
    images_resized, ages, test_size=0.2, random_state=42
)

# 3. Extract HOG features
def extract_hog_features(images):
    hog_features = []
    for img in images:
```

```
        features, _ = hog(img,
                          orientations=8,
                          pixels_per_cell=(16,16),
                          cells_per_block=(2,2),
                          visualize=True,
                          channel_axis=-1)  # For color images
        hog_features.append(features)
    return np.array(hog_features)

X_train_hog = extract_hog_features(X_train_img)
X_test_hog = extract_hog_features(X_test_img)

# 4. Standardize HOG features
scaler = StandardScaler()
X_train_hog_scaled = scaler.fit_transform(X_train_hog)
X_test_hog_scaled = scaler.transform(X_test_hog)
```

2.3 CNN Feature Extraction (VGG16)

Process:

- **Load and resize images** to $224 \times 224 \times 3$ (RGB), and extract age labels.
- **Split** the dataset into training and test sets.
- **Extract CNN features** from each image using a pretrained VGG16 model (without top layer, global average pooling).
- **Standardize** the extracted features (zero mean, unit variance) using statistics computed from the training set, then apply the same transformation to the test set.

Example code:

```
import numpy as np
import glob
import os
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from skimage.io import imread
from skimage.transform import resize
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input

# data_path = ... (see previous example)
# 1. Load and resize images
# images_resized = ...

ages = []
images_resized = []
img_size = (224, 224)   # VGG16 input size

for file in image_files[:100]:   # Use the first 100 images for demonstration
    age = int(os.path.basename(file).split('_')[0])  # Age is the first part of filename
    img = imread(file)
    img_resized = resize(img, img_size, anti_aliasing=True)
    ages.append(age)
    images_resized.append(img_resized)

```

```python
ages = np.array(ages)
images_resized = np.array(images_resized) #Convert lists to NumPy arrays for efficient numerical operations and compatibility with scikit-learn and deep learning frameworks.

# 2. Split into train/test before feature extraction
X_train, X_test, y_train, y_test = train_test_split(
    images_resized, ages, test_size=0.2, random_state=42
)

# 3. Extract CNN features using pre-trained VGG16
def extract_vgg_features(images):
    """
    Extract VGG16 features from a list/array of images.
    Each image must be 224x224x3 and pixel values in [0, 1].
    """
    model = VGG16(weights='imagenet', include_top=False, pooling='avg')
    features = []
    for img in images:
        # Preprocessing: rescale to [0,255], then preprocess as VGG expects
        img_batch = np.expand_dims(img * 255.0, axis=0)
        img_preprocessed = preprocess_input(img_batch)
        feature_vector = model.predict(img_preprocessed, verbose=0)
        features.append(feature_vector.flatten())
    return np.array(features)

print("Extracting VGG16 features for training set...")
X_train_vgg = extract_vgg_features(X_train)
print("Extracting VGG16 features for test set...")
X_test_vgg = extract_vgg_features(X_test)

# 4. Standardize VGG16 features
scaler = StandardScaler()
X_train_vgg_scaled = scaler.fit_transform(X_train_vgg)
X_test_vgg_scaled = scaler.transform(X_test_vgg)
```

Q&A: Why convert the image list to a NumPy array for CNN, but not for HOG?

2.4 Summary

Each feature engineering method provides different types of information for regression models. Comparing their results helps us understand which representation is most effective for face age estimation.

3 Linear Regression Model

Regression is a fundamental tool for modeling the relationship between a response variable Y and a set of predictors X. The general form of a linear regression model is:

$$Y = X\beta + \varepsilon$$

where

Y: The outcome (response) vector.

X: The design matrix of predictors.

β: The vector of coefficients to be estimated.

ε: The random error vector.

4 Regression Methods

4.1 Ordinary Least Squares (OLS)

4.1.1 Theory: OLS

Ordinary Least Squares (OLS) is the classical method for fitting a linear regression.

OLS finds the coefficients β that minimize the sum of squared errors between observed and predicted values:

$$\hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2$$

where

y_i: The observed response value for sample i.

$\mathbf{x}_i$: The feature vector for sample i (a row of X).

β: The vector of regression coefficients to be estimated.

$(y_i - \mathbf{x}_i^\top \beta)$: The residual (error) for sample i.

The closed-form solution to this minimization is:

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y$$

where X is the $n \times k$ feature matrix ($n \gg k$), and Y is the outcome vector.

Notes:

- OLS gives the Best Linear Unbiased Estimator (BLUE) under standard assumptions (linearity, constant variance, etc).
- For $(X^\top X)^{-1}$ to exist, the columns of X must be linearly independent (i.e., X is full rank $n \gg k$). If X is not full rank, OLS cannot uniquely determine β.

4.1.2 Model Evaluation: Mean Absolute Error (MAE)

To evaluate how well a regression model performs, we use the Mean Absolute Error (MAE) as the main metric:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

where y_i is the true value and $\hat{y}_i$ is the predicted value for sample i. MAE measures the average absolute difference between predictions and ground truth.

Q&A: Is a higher or lower MAE better?

example:
If one model has MAE = 10 and another model has MAE = 5, which model is better? Why?

4.1.3 OLS Example Code

The following code fits an OLS regression using VGG16 (CNN) features and computes MAE on the test set.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

vgg_model_ols = LinearRegression().fit(X_train_vgg_scaled, y_train)
vgg_pred_ols = vgg_model_ols.predict(X_test_vgg_scaled)
mae_ols = mean_absolute_error(y_test, vgg_pred_ols)
print("VGG16 Features OLS MAE:", mae_ols)
```

Listing 1: Example: Fit OLS regression on VGG16 features and compute MAE

Tip: You can use the same code with HOG or PCA features by changing the input feature matrices.

Q&A: Why is the OLS MAE so large?

In practice, when the number of features is large or some features are highly correlated, $X^\top X$ may be nearly singular. This can make OLS solutions unstable or undefined.

How to improve:

- Use regularization methods such as Ridge or Lasso regression to control overfitting and reduce the MAE.
- Increase the number of training samples if possible.
- Use or design features more closely related to age prediction (e.g., fine-tune a model on age-labeled data).

4.2 Ridge Regression (L2 Regularization)

4.2.1 Theory

Ridge regression adds an L_2 penalty (the sum of squared coefficients) to the OLS loss, which shrinks all coefficients towards zero but does not set any exactly to zero. This improves model stability, especially when predictors are highly correlated or the feature dimension is large.

$$\hat{\beta}_{\text{ridge}} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \lambda \sum_{j=1}^{k} \beta_j^2 \right\}$$

where $\lambda \geq 0$ is a tuning parameter controlling the strength of regularization.

The closed-form solution is:

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

4.2.2 Geometric Interpretation

- The L_2 penalty corresponds to a circular (or spherical) constraint region: $\sum_j \beta_j^2 \leq S$.
- The loss contours (ellipses) are unlikely to touch the boundary at the axes, so coefficients are typically shrunk toward zero but not set exactly to zero.
- **Ridge rarely yields exact zeros:** All predictors remain in the model, but their influence is reduced.

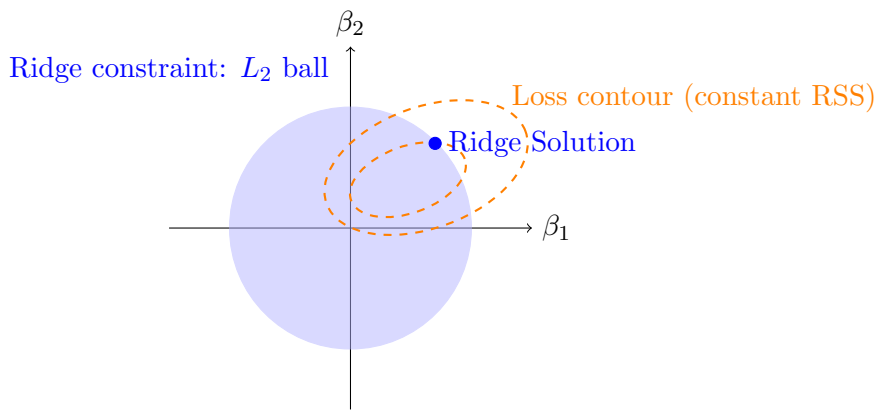

Figure 2: Ridge regression: The L_2 (blue) constraint region and loss contours (orange ellipses). The solution (blue dot) is where the smallest loss contour touches the constraint. This is rarely on the axes, so coefficients are usually nonzero.

Explanation:

- The blue circle represents all possible coefficient vectors β that satisfy the Ridge constraint $\|\beta\|_2 \leq S$.
- The orange dashed ellipses show contours of the squared error loss function.
- The optimal Ridge solution is at the intersection between the smallest (innermost) loss contour and the boundary of the constraint region.
- Because the L_2 ball is round, this intersection is almost never on the coordinate axes. As a result, Ridge regression shrinks coefficients but does not set them exactly to zero.

Why not zeros?

- The L_2 norm is smooth and the constraint region is round (no sharp corners), so the solution tends to be "shrunk" inside the feasible set but not at the axes.
- Ridge is best when many predictors have small, nonzero effects, and variable selection is not the goal.

4.3 Lasso Regression (L1 Regularization)

4.3.1 Theory

Lasso regression adds an L_1 penalty (the sum of absolute coefficients) to the OLS loss. This both shrinks coefficients and can set some exactly to zero, effectively performing variable selection:

$$\hat{\beta}_{\text{lasso}} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \lambda \sum_{j=1}^{k} |\beta_j| \right\}$$

where $\lambda \geq 0$ controls the regularization strength.

4.3.2 Geometric Interpretation

- The L_1 constraint region, defined by $\sum_j |\beta_j| \leq S$, forms a diamond shape in two dimensions (and a polyhedron in higher dimensions).
- The loss contours (ellipses) frequently touch the constraint boundary at the axes, which allows Lasso to set some coefficients β_j exactly to zero. This leads to sparse models and automatic feature selection.

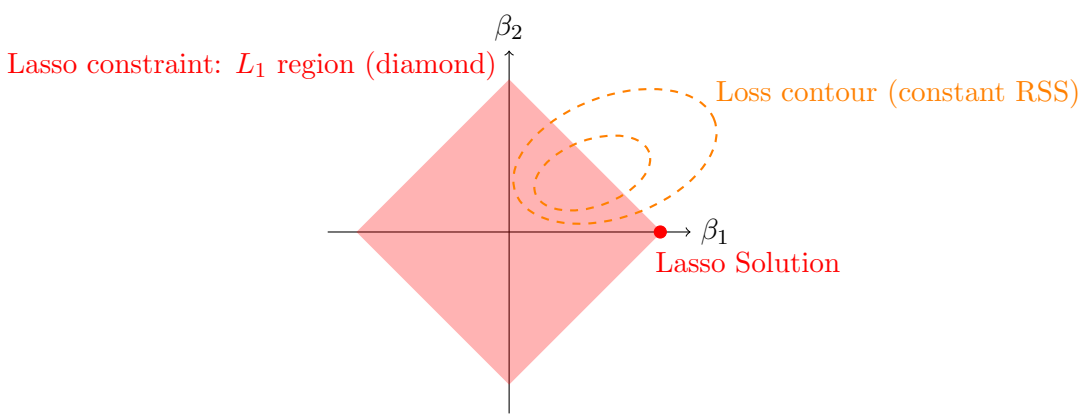

Figure 3: Lasso regression: The L_1 (red) constraint region and loss contours (orange ellipses). The solution (red dot) is at a corner of the constraint, which lies on the β_1 axis. This produces sparse solutions with some coefficients exactly zero.

Explanation:

- The red diamond shows all possible coefficient vectors β that satisfy the Lasso constraint $\|\beta\|_1 \leq S$.
- The orange dashed ellipses are contours of equal squared error loss.
- The optimal Lasso solution is often located at a corner of the diamond (on an axis), so one or more coefficients are exactly zero (feature selection).
- Lasso regression is therefore well suited for sparse modeling and automatic variable selection.

5 Why Regularization and Zeros Matter

Regularization (such as Lasso or Ridge) is used to address overfitting and improve model interpretability by penalizing large or unnecessary coefficients.

- **Why do we need regularization?**
  Overfitting occurs when a model fits noise in the training data, rather than general patterns. For example, if you memorize the answers to specific practice questions, you may do well on those but struggle with new questions since the model does not generalize.

- **How does regularization work?**
  Regularization adds a penalty term to the loss function:

  - **Lasso (L_1)** penalty: $\sum_j |\beta_j|$
  - **Ridge (L_2)** penalty: $\sum_j \beta_j^2$

  The total loss is minimized as:

  $$\text{Total Loss} = \text{MSE} + \lambda \cdot \text{Penalty}$$

  where λ controls the strength of the penalty (*e.g.*, $\lambda = 0$ is no penalty, large λ may lead to underfitting).

- **Why are zeros useful? When is zero good?**

  - Setting coefficients to zero eliminates uninformative predictors, resulting in a simpler and more interpretable model through automatic feature selection.
  - Lasso is especially useful when only a few predictors are truly important and most should be excluded.
  - In high-dimensional problems (such as those involving CNN features), forcing some coefficients to zero can help prevent overfitting and enhance interpretability.

6 Model Evaluation and Selection

The performance of each model is evaluated using MAE on the test set. Cross-validation can be used to choose optimal regularization parameters.

Example: Ridge regression with CNN (VGG16) features

```
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_absolute_error

vgg_model_ridgecv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
vgg_model_ridgecv.fit(X_train_vgg_scaled, y_train)
vgg_pred_ridgecv = vgg_model_ridgecv.predict(X_test_vgg_scaled)
mae_ridgecv = mean_absolute_error(y_test, vgg_pred_ridgecv)
print("VGG16 Features RidgeCV Best Alpha:", vgg_model_ridgecv.alpha_)
print("VGG16 Features RidgeCV MAE:", mae_ridgecv)
```

Listing 2: Ridge regression with cross-validated alpha (CNN features)

Example: Lasso regression with CNN (VGG16) features

```python
from sklearn.linear_model import LassoCV
from sklearn.metrics import mean_absolute_error

vgg_model_lassocv = LassoCV(alphas=[0.001, 0.01, 0.1, 1, 10], cv=5, max_iter=5000)
vgg_model_lassocv.fit(X_train_vgg_scaled, y_train)
vgg_pred_lassocv = vgg_model_lassocv.predict(X_test_vgg_scaled)
mae_lassocv = mean_absolute_error(y_test, vgg_pred_lassocv)
print("VGG16 Features LassoCV Best Alpha:", vgg_model_lassocv.alpha_)
print("VGG16 Features LassoCV MAE:", mae_lassocv)
```

Listing 3: Lasso regression with cross-validated alpha (CNN features)

Exercise:

Use the UTKFace dataset to train and evaluate OLS, Ridge, and Lasso regression models with CNN (VGG16) features.

- Run the above code to compute the MAE (mean absolute error) for each model.
- Try different values of α in Ridge and Lasso (e.g., change the `alphas` list) to see which regularization strength works best on the UTKFace data.
- Which model gives the lowest MAE? Did regularization (Ridge/Lasso) improve performance compared to OLS?
- Explain why tuning the regularization parameter α can affect model performance, especially when working with high-dimensional features.

6.1 Comparison: Ridge vs. Lasso

Ridge Regression	**Lasso Regression**
Shrinks all coefficients but typically does not set any exactly to zero.	Can set some coefficients exactly to zero, resulting in sparse solutions (feature selection).
Circle constraint (L_2 ball), solution rarely lands on the axes.	Diamond constraint (L_1 region), solution often lands on the axes.
Best when many predictors have small effects.	Best when only a few predictors are important (sparse).

Which is better?

7 Conclusion

- Use regularization (Ridge/Lasso) when feature dimension is large or data is noisy.
- Ridge generally works well when all features contribute a little; Lasso is preferred for automatic feature selection.
- Model selection should be based on validation MAE and domain needs.